TokenShare: Sharing Tokenized Intelligent Capacity for Verifiable Useful Work

Hongzhan Tu Zhizhou Wang

July 2, 2026

Abstract

Large useful tasks increasingly require more than a single model invocation. They consume model tokens, tool calls, local compute, formal checkers, human attention, retries, and verification effort. Individually, these resources are fragmented across people, machines, organizations, and agents; collectively, they form a latent pool of intelligent capacity. This paper proposes *TokenShare*: the concept of turning fragmented tokenized capacity into verifiable, mergeable, and accountable contributions to a larger task.

The distributed system in this paper is an instrument for making that concept concrete. It represents a task as a recursively expandable task DAG, assigns bounded units through leases and attempts, persists submissions and artifacts, verifies outputs through task plugins, merges accepted child results, and records contribution accounting in an append-only event ledger. We instantiate TokenShare with two real proof-of-concept domains: integer factorization with deterministic range verification and Lean formal-proof production with a fixed local Lean/lake checker environment. We also evaluate an experiment-scoped AI API executor that records raw model output, parsed output, parse failure, usage, cost, latency, and provider provenance. Across 17 default experiment runs, TokenShare exercises end-to-end useful-work sharing, failure isolation, mechanism ablation, and cross-domain generality, producing 1215 events and 510 artifacts. The evidence supports the central claim: shared tokenized capacity becomes useful only when the system can decompose work, verify contributions, merge accepted evidence, and replay why each contribution counted.

1 Introduction

The simplest intuition behind modern AI work is that more attempts can buy more capability. A model may solve a problem after additional tokens, a tool call may expose missing evidence, a proof checker may reject a plausible answer, and a human reviewer may repair an ambiguous output. These attempts are not free. They consume tokenized units of intelligent capacity: model tokens, API calls, local compute, formal-checker time, domain expertise, and human attention.

Many useful tasks need more capacity than one user, model session, machine, or organization can conveniently spend. Scientific computation, formal reasoning, benchmark construction, data transformation, annotation, and public-good knowledge work all require decomposition, bounded execution, verification, repair, and integration. At the same time, useful capacity is widely scattered: idle local machines, unused API credits, personal agents, laboratory servers, tool environments, and human reviewers often hold small but meaningful pieces of the required work.

TokenShare is the proposal that those fragmented units can be shared as contributions to a common useful task. The essential idea is not merely parallel execution. A shared token must become a contribution: it must be attached to a task unit, bounded by an execution request, preserved as evidence, checked by a task-specific verifier, selected into canonical state, merged with other accepted evidence, and recorded so that later participants can audit why it mattered. The distributed-system machinery in this paper exists to enforce that conversion from spent capacity to accepted contribution.

This paper makes three main **contributions**:

- We formulate TokenShare as a shared-capacity concept: fragmented model tokens, tool calls, compute, checker time, and human attention can be converted into verifiable contributions to decomposable useful work.

- We show how a distributed protocol can operationalize this concept through task DAGs, leases, artifact-backed submissions, plugin-owned verification, canonical selection, merge, contribution accounting, and replayable evidence.
- We evaluate the concept with real factorization and Lean proof plugins, failure injection, mechanism ablation, cross-plugin generality, and a real AI API executor profile, clarifying what each experiment demonstrates about useful token sharing.

2 Motivation Towards Shared Token Economics

2.1 Background

Tokens as Units of Intelligent Capacity In TokenShare, a "token" is broader than a language-model token. It is any bounded unit of intelligent capacity that can be spent toward a task: a model call, a tool invocation, a local computation slice, a checker run, a repair attempt, or a unit of human review. The point of sharing tokens is not to pool raw expenditure for its own sake, but to make small resource contributions composable into a larger verified output.

The Price of Useful Sharing Sharing capacity is easy when correctness does not matter; it is hard when the final artifact must be trusted. A useful shared contribution must answer four questions: what resource was spent, what task unit it addressed, what evidence it produced, and why that evidence was accepted or rejected. TokenShare treats those questions as first-class protocol data rather than after-the-fact narration.

2.2 Motivations

The motivating question is therefore: *Can a community share tokenized intelligent capacity in a way that produces useful work rather than unstructured effort?*

TokenShare answers by separating the concept from the tool that realizes it. Conceptually, TokenShare is a contribution economy for useful intelligent work: shared resources should become accepted contributions only after they are bounded, evidenced, verified, and mergeable. Operationally, the prototype uses a local distributed-systems design to enforce those properties. The protocol core maintains task units, dependency edges, leases, attempts, artifact references, verification reports, canonical output bindings, expansion decisions, merge records, contribution records, and accounting records.

A task plugin defines what counts as a valid contribution for a domain; an executor merely spends capacity and returns evidence. This boundary is central to TokenShare. A model-backed contributor may spend tokens, but raw text alone is evidence rather than contribution. A range worker may spend compute, but a false factor claim is rejected expenditure rather than useful work. A Lean proof attempt may spend checker time, but only a checker-backed proof artifact can count. TokenShare is the bridge between resource expenditure and accepted contribution.

3 The Design Philosophy

3.1 Problem Formulation

We consider a useful root task T and a population of contributors who can spend heterogeneous tokenized resources:

$$\mathcal{K} = \{k_1, k_2, \ldots, k_m\},$$

where a token k_i may be a model call, local computation interval, checker invocation, tool call, or human review action. TokenShare is concerned with when a spent token becomes a contribution. We model an accepted contribution as

$$c = (u, k_i, a, e, v, m),$$

where u is the task unit, a is the execution attempt, e is the produced evidence artifact, v is the verification result, and m is the merge or accounting relation that explains how the contribution affects the larger task.

The implementation realizes this concept as a protocol instance

$$\mathcal{P}_T = (G_T, \mathcal{A}_T, \mathcal{E}_T, \mathcal{R}_T),$$

where G_T is the task DAG, $\mathcal{A}_T$ is the content-addressed evidence set, $\mathcal{E}_T$ is the append-only event log, and $\mathcal{R}_T$ freezes the plugin and executor descriptors used by the run. The distributed system is therefore not the definition of TokenShare; it is the evidence machine that makes shared contributions assignable, checkable, mergeable, and replayable.

Recursive decomposition controls the granularity at which tokens can be shared. A plugin-owned split strategy may transform an already verified canonical output bundle into

$$\mathrm{Split}_p(u, \hat{Y}_u) \to (\Delta G, \mathrm{MergePlan}),$$

where ΔG contains child units and dependencies, and the merge plan records the required child output slots. This is the key design move: TokenShare can invite more capacity into the system only when the resulting units have explicit inputs, expected outputs, verification rules, and merge obligations.

3.2 Desiderata of TokenShare Protocol

Desideratum I. ***Decomposability:*** *A TokenShare task should be expressible as a structured task tree or directed acyclic graph whose executable units have clear inputs, outputs, dependencies, and merge relations.*

A TokenShare task must be decomposable into units small enough for shared contributors to execute, but structured enough for the result to rejoin the parent task. Decomposition is therefore not a free-form plan emitted by an executor; it is a versioned plugin decision recorded through a structured proposal and an expansion event batch. In the factorization plugin, decomposition partitions a candidate divisor domain into bounded range units. In the Lean plugin, decomposition is produced by a deterministic Lean-side helper that emits a split certificate and a merge skeleton. In both cases, decomposition creates shareable contribution opportunities without giving contributors authority over the task graph.

Desideratum II. ***Local Verifiability:*** *Each executable subtask should include an explicit verifier that can check whether its submitted output is valid before that output is accepted, merged, or counted as eligible contribution evidence.*

Each candidate output must be verifiable before it becomes a contribution. TokenShare separates parsing, structural checks, domain verification, and canonical selection. For factorization, the plugin verifier rechecks whether a claimed factor divides the target and whether a no-factor range claim is consistent with its bounded domain. For Lean proof production, the validator accepts only checker reports backed by a fixed local Lean/lake environment, generated source, stdout/stderr logs, proof artifacts, and an environment reference. A raw model response or executor submission is therefore evidence, not authority.

Desideratum III. ***Extendability:*** *The protocol should provide a reusable coordination skeleton while allowing each task family to plug in task-specific schemas, verifiers, merge rules, cost models, and contributor requirements.*

The concept should survive beyond a single task family. TokenShare implements extendability through frozen plugin descriptors and executor descriptors. A plugin supplies task types, schemas, parser policy, split strategy, validator policy, and merge policy. An executor supplies a transport or local execution boundary and returns artifacts. This makes the same contribution lifecycle coordinate arithmetic range search and Lean proof checking while keeping divisibility rules and proof-checking rules outside the shared protocol.

Desideratum IV. ***Scalability:*** *As more tokens, agents, workers, or computational resources become available, the protocol should convert that additional capacity into faster completion, higher quality, broader verification, or stronger fault tolerance.*

The relevant scaling unit is not only the number of workers, but the amount of independently checkable evidence the system can admit without corrupting accepted state. TokenShare therefore scales through smaller typed artifacts, explicit lease and attempt records, deterministic merge gates, and replayable event batches. Additional tokenized capacity is useful only when its output can be parsed, verified, linked to an expected slot, and merged under a recorded policy.

Desideratum V. ***Autonomy:*** *The protocol should advance tasks through assignment, execution, validation, verification, merging, timeout recovery, reassignment, and contribution accounting according to predefined rules with minimal continuous manual coordination.*

The protocol should advance routine lifecycle transitions from recorded facts rather than conversational memory. A request produces a submission; a submission produces a verification report; verified candidates may compete for a single canonical output bundle; canonical outputs may trigger completion or expansion; merge records and expected-output resolution produce parent completion; eligible canonical contributions enter an accounting batch. The autonomy claim is therefore conceptual and operational: shared tokens can be converted into work units without constant manual coordination, and the system can later explain why a task progressed, blocked, or rejected an output.

4 The TokenShare Protocol

The TokenShare protocol is the mechanism that turns the concept of shared tokenized capacity into executable work. Its purpose is to decide when resource expenditure becomes accepted contribution. The implementation has three boundaries: protocol objects and state transitions, task plugins, and executors. Protocol objects define the reusable contribution lifecycle; plugins define task semantics; executors spend resources and produce submissions under a request contract. The event ledger and artifact store connect the three layers by making every authority-bearing fact persistent.

4.1 Overall Picture

Figure 1 shows how the concept is implemented. A root task enters the system as a typed request for shared work. The protocol engine turns that request into shareable task units, records registry snapshots, schedules ready units through leases, records execution requests and submissions, invokes verification and canonical selection, accepts plugin-owned expansion proposals, resolves merge tasks, and records contribution accounting. SQLite is a materialized index over the append-only event ledger rather than the source of authority.

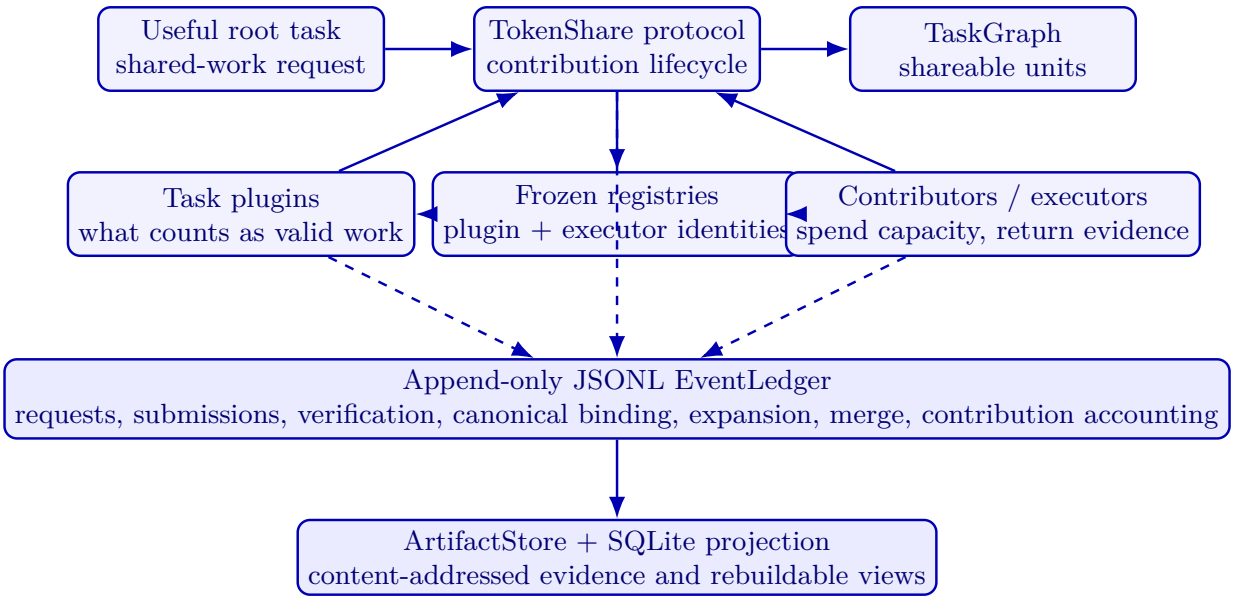

Figure 1: TokenShare as a concept implemented by a distributed protocol. Contributors and executors spend tokenized capacity; plugins decide what counts as valid work in a domain; the protocol records the contribution lifecycle that makes accepted work shareable, mergeable, and auditable.

4.2 Task Registration, Decomposition, Assignment, and Commitment

4.2.1 Task Registration

A registered task is a typed protocol object rather than an informal request. The root `TaskUnit` stores a `TaskSpec`, plugin identity, required outputs, dependency state, and artifact references. The input body is persisted through a content-addressed artifact reference, and task creation is recorded as an event. This gives later verification and replay a stable object identity:

$$\text{Root}(T) = (\text{task\_id}, \text{unit\_id}, \text{plugin\_id}, \text{input\_ref}, \text{required\_outputs}).$$

4.2.2 Recursive Task Decomposition

TokenShare expands only from accepted evidence. A candidate output first passes parser and plugin-domain validation. If it is selected as the canonical output bundle for a unit, the plugin split strategy may derive a `DecompositionProposal` and a `MergePlan`. The accepted expansion is committed as a batch:

$$\text{proposal} \rightarrow \text{decision} \rightarrow \text{merge plan} \rightarrow \text{child units} \rightarrow \text{TASK\_EXPANDED}.$$

This order prevents child tasks from appearing as authoritative state before the proposal, decision, and merge requirements that justify them.

4.2.3 Task Assignment and Commitment

Assignment is represented by leases and attempts. A scheduler selects ready units under a protocol configuration, a `LeaseManager` records an active lease with a fencing token, and an execution request binds an attempt to the task, unit, lease, executor descriptor, input artifacts, and optional prompt package. A submission advances an attempt only when it matches the current task, unit, attempt, lease, and fencing token. This gives TokenShare the commitment property it needs for auditability: an executor cannot retroactively make a submission authoritative for a different leased unit.

4.3 Execution, Verification, Recursive Merge, and Contribution Accounting

4.3.1 Client-Side Execution and Local Validation

Executors operate behind a uniform request/submission interface. A deterministic local executor may compute a bounded factor-search result; the AI API executor may call a model and persist raw provider output; the Lean checker path may generate checker reports and proof artifacts. In every case, the executor returns artifact references and provenance, not protocol acceptance. Local validation is useful as an executor-side guardrail, but accepted protocol state begins only after the recorded submission is parsed, verified, and selected through the protocol lifecycle.

4.3.2 Server-Side Verification

Server-side verification is layered. The protocol checks artifact presence, schema shape, required-output coverage, and attempt linkage. The plugin then performs domain validation. Factorization verifies range outputs by deterministic arithmetic recheck. Lean validation requires a real checker report with generated source, proof artifact, stdout/stderr logs, and an `EnvironmentRef`. Verification produces an auditable report; canonical selection is a separate event that binds one eligible output bundle for a task unit.

4.3.3 Recursive Result Merging

Merge is represented as another protocol task rather than an implicit post-processing step. A `MergePlan` records required slots, expected child output refs, schema requirements, and the plugin merge policy. A merge task becomes resolvable only when all required slots have canonical child outputs. The plugin merge policy constructs the candidate parent output, the protocol records a `MergeRecord`, and expected-output resolution determines whether the parent can complete or must remain unresolved.

4.3.4 Contribution Accounting

TokenShare's current accounting layer is evidence-derived. Contributions are derived from canonical completion, accepted expansion, and accepted merge facts. A root-level accounting batch consumes the complete eligible contribution set, writes a `SettlementRecord` as the current implementation artifact, persists entries for audit, and advances the corresponding contribution records to accounted. The useful invariant is idempotence: a completed root can be accounted for once from the same event and artifact evidence, and replay can inspect the batch without re-executing the task.

5 A Robust Fault-Tolerance Mechanism

Fault tolerance in TokenShare is expressed as evidence isolation. A failed executor call, malformed model response, invalid domain output, expired lease, or incomplete merge should leave an auditable trace without contaminating canonical state. The current prototype evaluates this principle through parser isolation, deterministic plugin verification, lease expiry and requeue, bounded verifier budgets, all-required merge gates, and idempotent accounting batches.

5.1 Failure Model

The experiment suite distinguishes five implemented failure classes:

- **Offline**: an assigned unit stops producing valid submissions and must be recovered through the lease path.
- **Slow**: progress is delayed relative to the lease and scheduling configuration.
- **Executor error**: the execution boundary records a failed call rather than a candidate output.
- **Invalid output**: a structured candidate reaches verification but fails the plugin validator.
- **Late submission**: a submission arrives after the relevant lease authority has expired.

The suite also injects task-specific factorization failures: invalid factors, false no-factor claims, raw-only model output, expired lease recovery, and no-factor recheck budget exhaustion.

5.2 Task State and Idempotent Work Units

TokenShare separates task, lease, and attempt state. A `TaskUnit` records coarse lifecycle state and dependency readiness. A `Lease` records authority to work on a unit for a bounded interval. An `Attempt` records execution progress and the submission/verification/canonical path. Idempotence is enforced at the event boundary: retries must either match the same logical fact or be recorded as distinct evidence rather than overwriting prior state.

5.3 Lifecycle State Machine

The implemented lifecycle uses three state machines:

$$\begin{aligned}
\text{TaskUnit}: &\quad \textsc{Created} \to \textsc{Ready} \to \textsc{Processing} \to \textsc{Completed},\\
\text{Lease}: &\quad \textsc{Active} \to \{\textsc{Released}, \textsc{Expired}, \textsc{Revoked}\},\\
\text{Attempt}: &\quad \textsc{Created} \to \textsc{Running} \to \textsc{Submitted} \to \{\textsc{Verified}, \textsc{Rejected}\} \to \textsc{Canonical}.
\end{aligned}$$

This split prevents a timeout from being confused with output rejection, and prevents a verified but non-canonical attempt from being treated as completed work.

5.4 Failure Detection

Failure detection is currently ledger-driven. Heartbeats extend evidence of lease activity; lease expiry returns the unit to a schedulable state when retry policy allows; executor errors and parse failures are persisted as artifacts or submissions; invalid outputs are rejected by verification reports. The important property is that each failure is observed at a named boundary, so later metrics can attribute the recovery to parser, verifier, lease, budget, or merge behavior.

5.5 Adaptive Redundant Assignment

The prototype focuses on mechanism necessity rather than adaptive redundancy. Experiment 3 disables one boundary at a time: verification, parser policy, requeue, all-required merge gate, and slot-integrity checking. The resulting degradations show why redundancy alone would be insufficient: if malformed raw output can bypass parsing or if a merge can ignore missing slots, additional attempts only produce more untrusted evidence.

5.6 Straggler Mitigation and Lease Recovery

The current recovery path is lease-based. A unit with an expired lease can be requeued, while prior attempts and artifacts remain in the ledger for audit. In the experiments, worker-crash recovery is demonstrated by preserving the failed evidence and completing through a later valid attempt. This ties the recovery claim to implemented lease, retry, and audit machinery.

5.7 Verification Boundaries

Verification is currently performed at explicit protocol boundaries and delegated to deterministic plugin validators. This is a strength of the two proof-of-concept applications: factorization provides arithmetic recheck, and Lean provides an external proof checker. The protocol records verification reports in a form that preserves validator identity, inputs, outputs, logs, and acceptance status, so the reported experiments can be audited through domain-specific evidence rather than informal agreement.

5.8 Semantic Quality Aggregation

Semantic quality aggregation is not used as experimental evidence in this paper. Its relevance is architectural: TokenShare's parser, verification-report, canonical-selection, and artifact interfaces could host probabilistic or review-based validators for future weak-verification domains. The present evaluation deliberately uses tasks with checkable acceptance conditions so that protocol failures cannot be hidden behind subjective quality scores.

5.9 Contribution Eligibility and Delayed Accounting

The implemented accounting layer records contribution eligibility after canonical completion, accepted expansion, or accepted merge. Accounting is delayed until the root completion path can identify the complete eligible contribution set. This is sufficient for the paper's mechanism claim: accounting is derived from accepted protocol facts, not from executor self-reporting.

5.10 Dispute Resolution

Dispute resolution appears in the prototype as boundary rejection rather than a separate adjudication workflow. The useful pattern is:

1. **Parse isolation**: raw-only or malformed output produces a parse-failure artifact.
2. **Verification rejection**: structured but wrong output produces a rejected verification report.
3. **Canonical exclusion**: verified losers remain evidence but do not bind canonical state.

4. **Merge blocking**: incomplete or slot-inconsistent child output sets do not complete the parent.

These outcomes are enough to explain why invalid evidence remains visible without becoming accepted evidence.

5.11 Control-Plane Fault Tolerance

The control plane is event-sourced. Task graph changes, leases, attempts, requests, submissions, verification reports, canonical bindings, expansion batches, merge records, contribution records, and accounting records are stored as append-only events and artifact references. This gives the prototype three properties:

- **Recoverability**: materialized views are rebuilt from event and artifact evidence.
- **No double accounting**: root accounting is a batch fact over a complete eligible contribution set.
- **Auditable provenance**: accepted outputs link back to requests, submissions, checker evidence, merge records, and environment or provider metadata.

5.12 Fault-Tolerance Summary

TokenShare's fault-tolerance claim is therefore narrow and testable: failures may produce extra events, rejected artifacts, parse-failure reports, expired leases, or inconclusive ablation runs, but they should not silently alter canonical output. The reported experiments evaluate precisely this property, including an ablation case where removing verification creates canonical pollution and thereby demonstrates the necessity of the boundary.

6 Proof of Concept Applications

The two applications show how the TokenShare concept survives different meanings of "useful work." In factorization, a useful contribution is bounded arithmetic search with deterministic verification and strict all-required merge slots. In Lean proof production, a useful contribution is a proof candidate or subproof accepted by a fixed local checker with proof artifacts, environment references, and split/merge proof evidence. Figure 2 summarizes the common loop by which tokenized capacity becomes accepted contribution in both applications.

6.1 Application I: Large Integer Factorization

The first proof-of-concept application is the `factorization@0.1.0` plugin. Its role is not to compete with specialized number-theory software; it supplies a compact workload in which the protocol can observe typed decomposition, bounded execution, deterministic verification, all-required merge, contribution accounting, and canonical-state safety.

Task specification. The registered task is

$$\mathcal{S}_{\text{fac}} = (n, \text{requested\_output}, \text{params}, \text{plugin\_version}),$$

where n is stored as a decimal string and the requested output is a prime factorization result. A range child accepts only `factorization.range_result.v1`. The core arithmetic check is

$$V_{\text{fac}}(n, d) = \begin{cases} 1, & \text{if } 1 < d < n \text{ and } n \bmod d = 0, \\ 0, & \text{otherwise.} \end{cases}$$

and no-factor claims are also rechecked against their assigned closed interval. The plugin merge policy emits a final `PrimeFactorizationResult` only when the required evidence is complete.

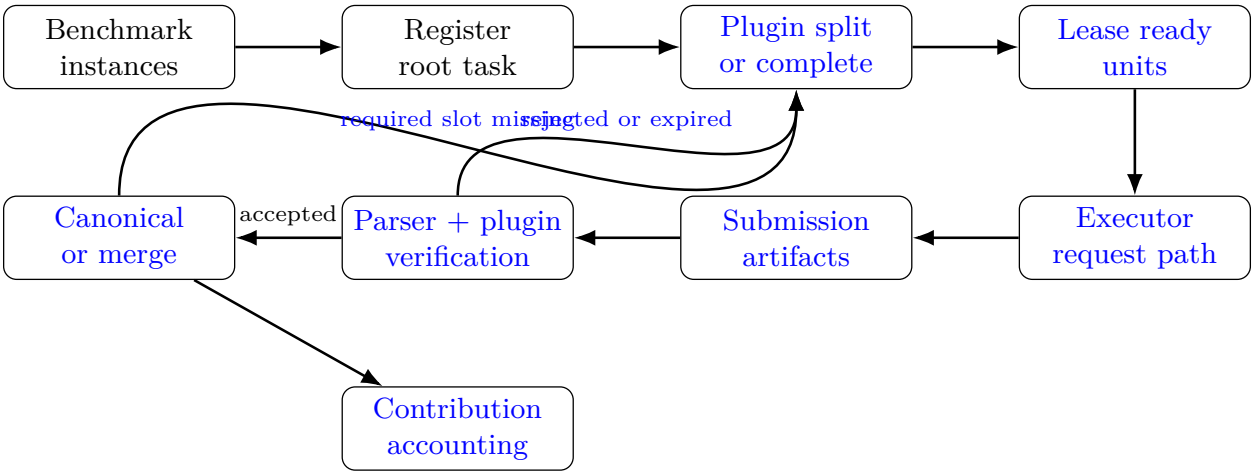

Figure 2: Common proof-of-concept workflow. Each benchmark instance is registered as a root task, optionally expanded by a plugin-owned split strategy, executed through an executor boundary, verified by a task-specific checker, and recorded through canonical, merge, contribution, and accounting events.

Decomposition. The plugin split strategy partitions the candidate divisor interval $[2, \lfloor\sqrt{n}\rfloor]$ into ranges

$$I_j = [a_j, b_j],$$

with a coverage proof showing no gaps and no overlaps. Each range unit receives a bounded instruction:

$$u_j = (n, [a_j, b_j], \text{coverage\_id}, \text{child\_index}).$$

The executor may search only inside that interval and may not create child tasks, invent output schemas, or claim the final factorization.

Experimental workflow. Experiment 1 runs small-prime, prime-range, semiprime-range, and extended semiprime fixture cases. Experiment 2 injects invalid factor, false no-factor, raw-only parse failure, expired lease, and verification-budget failures. Experiment 3 disables one boundary per run to show controlled degradation.

Baselines and metrics. The primary baseline is not a faster factorization algorithm but the same fixture lifecycle with selected protocol boundaries removed. Metrics therefore focus on final correctness, completion rate, canonical pollution, accounting success, event footprint, and artifact footprint.

6.2 Application II: Lean Formal Proof Production

The second proof-of-concept application is the `lean_proof@0.1.0` plugin. Lean supplies a high-value verification boundary: a proof artifact is accepted only if a fixed local Lean/lake environment checks it and the resulting checker evidence is persisted. This makes Lean a strong test for whether TokenShare can coordinate a domain whose correctness is external to the protocol core.

Task specification. The registered task is

$$\mathcal{S}_{\text{Lean}} = (s, \mathcal{I}, \Omega, \Pi),$$

Table 1: How the two applications instantiate TokenShare.

Component	Large integer factorization	Lean formal proof production
Root task	Factor a small integer fixture, including prime and semiprime cases.	Check a structured Lean theorem payload under fixed imports and environment Ω.
Task unit	`root`, `factor_integer`, `factor_search_range`, or `factorization_merge`.	Root theorem, child theorem, or merge-proof unit.
Decomposition	`candidate_range_partition.v1` covers $[2, \lfloor\sqrt{n}\rfloor]$ with required range slots.	Lean-side deterministic split helper emits certificate-backed child goals and merge skeletons.
Submission	Structured range-result artifact or parser-failure evidence from raw output.	Schema-bound proof candidate, checker report, proof artifact, and environment evidence.
Verifier	Deterministic arithmetic recheck and all-required merge policy.	Fixed local Lean/lake checker with persisted stdout, stderr, generated source, and `EnvironmentRef`.
Recovery	Parser isolation, invalid-output rejection, expired lease requeue, and merge blocking.	Rejected proof candidates remain evidence; accepted child proofs feed a checker-validated merge proof.
Main metrics	Final correctness, canonical pollution, completion, accounting, event count, artifact count.	Real checker evidence, environment completeness, decomposition/merge lifecycle coverage.

where s is a structured theorem statement, $\mathcal{I}$ is the allowed import set, Ω is the fixed Lean/lake environment, and Π is the decomposition policy. A proof candidate must bind to the theorem payload digest. The checker predicate is

$$V_{\text{Lean}}(s, y, \Omega) = \begin{cases} 1, & \text{if the project builds successfully in } \Omega, \\ 0, & \text{otherwise.} \end{cases}$$

and an accepted report must include logs, generated source, proof artifact, and `EnvironmentRef`.

Recursive decomposition. Each Lean task unit is represented by a structured theorem payload rather than a natural-language goal. Direct proof and decomposition/merge paths are both supported in the fixture suite. A decomposition result is accepted only if the Lean-side helper emits a supported split certificate:

$$u = (s, \mathcal{I}, \Omega, \text{payload\_digest}, \Pi),$$

and a proof candidate may enter the protocol only as one of the following artifact-backed forms:

$$y_u \in \{\textsc{DirectProof}(p_u), \textsc{ChildProof}(p_i), \textsc{MergeProof}(p_{\text{root}})\}.$$

The merge policy reads accepted child proof artifacts, constructs a root merge proof, and invokes the checker again.

Experimental workflow. Experiment 4 runs a factorization semiprime lifecycle, a Lean direct proof fixture, and a Lean decomposition/merge fixture under the same experiment runner. The Lean adapter first performs a real preflight check, then reports checker evidence and environment completeness.

Recovery and accounting. Lean proof evidence enters the same accounting path as factorization evidence: accepted canonical proof artifacts and accepted merge facts can produce eligible contribution records, and root completion can trigger an accounting batch. Invalid proof candidates remain auditable checker evidence but do not bind canonical state.

Baselines and metrics. The Lean cases are evaluated by real checker evidence, environment-reference completeness, canonical pollution, event footprint, artifact footprint, and whether direct and decomposition/merge paths complete through the shared protocol lifecycle.

7 Experiments

7.1 Evaluation Objective

The experiments evaluate whether TokenShare works as a concept, not whether the prototype is an optimized factorization engine or theorem prover. Each experiment asks a different question about shared tokenized capacity: Can it be turned into a complete useful output? Can bad contributions be rejected without polluting shared state? Are the protocol boundaries necessary? Can the same sharing mechanism survive a change of task domain?

The implementation supplies the measurement instrument. We organize the evaluation into four experiments. Experiment 1 tests whether a real factorization plugin can convert shared execution units into a verified final result. Experiment 2 injects invalid or incomplete submissions to test whether rejected expenditure remains evidence rather than contribution. Experiment 3 disables individual protections to test whether TokenShare depends on parser ownership, verification, requeue, merge gates, and slot integrity. Experiment 4 tests whether the same contribution lifecycle generalizes from arithmetic search to real Lean proof production. We then report an auxiliary contributor-heterogeneity simulation that compares deterministic local, real model-backed, and malformed contributors on the same semiprime task.

Table 2: Evaluation groups as evidence for the TokenShare concept. The AI executor profile is treated as an auxiliary contributor-heterogeneity simulation rather than a separate experiment.

Group	Setting	Conceptual claim
Experiment 1	Factorization lifecycle	Shared bounded compute can become a verified final contribution through decomposition, execution, verification, merge, contribution recording, and accounting.
Experiment 2	Failure recovery	Invalid factors, false no-factor claims, parse failures, crashes, and verification-budget failures are isolated or recovered without canonical pollution.
Experiment 3	Mechanism ablation	Removing verifier, parser policy, requeue, merge gate, or slot-integrity checks breaks the conversion from spent capacity to accepted contribution.
Experiment 4	Cross-plugin generality	TokenShare is not tied to one task: arithmetic search and formal proof production share the same contribution lifecycle while preserving plugin-owned semantics.
Auxiliary simulation	Heterogeneous contributors	Model tokens, local compute, and malformed outputs can enter the same system, but only verified evidence becomes useful contribution.

7.2 Metrics and Status Definitions

We report metrics at two levels. Task-level metrics tell us whether the useful work was actually done: final correctness, parser success rate, parsed output count, and parse failure count. TokenShare-level metrics tell us whether spent capacity became trustworthy contribution: completion rate, canonical pollution count, accounting success, real checker evidence, event count, artifact count, provider attempts, retry count, token usage, cost estimate, and latency. We use *passed* for a run that satisfies the expected property, *failed* for an unexpected violation, *blocked* for an unavailable runtime dependency, and *inconclusive* for an intentional ablation run in which disabling a protection causes the expected degradation.

7.3 Reproducibility and Setting Artifacts

Before reporting results, we regenerated the experiment output tree from a clean `outputs/experiments` directory. The settings are not reconstructed from the prose of this paper; they are persisted next to the results. This is important for TokenShare because a contribution economy must make the conditions under which evidence was

produced visible: seed, fixture identity, plugin and executor identities, failure profile, ablation mode, request limits, parser policy, provider selection digest, and artifact paths are part of the audit surface.

Table 3: Regenerated setting and result artifacts used by the experiment section.

Artifact	Role	Contents used in the paper
`outputs/experiments/phase8_experiment_settings.json`	Experiment 1–4 settings	Seed 1, 17 experiment cases, fixture names, plugin versions, expected outputs, fault profiles, ablation modes, profile digests, and per-run manifest paths.
`outputs/experiments/phase8_suite_report.json`	Experiment 1–4 suite result	Total, passed, inconclusive, failed, and blocked run counts; summary and settings paths; local AI config digest without secrets.
`outputs/experiments/phase8_suite_summary.csv`	Paper table source	Per-case status, correctness, canonical pollution, completion, settlement, real checker evidence, event count, and artifact count.
`outputs/experiments/runs/*/run_manifest.json`	Run-level setting/evidence link	Frozen plugin and executor descriptors, seed, clock semantics, event-log ref, artifact root, status, and replayable run identity.
`outputs/experiments/ai_profile_real/ai_profile_settings.json`	Auxiliary AI-profile settings	Real-transport flag, request limits `max_tokens=1024` and `timeout_seconds=30`, JSON-mode parser policy, semiprime range inputs, config digest, entry metadata, and strict-arithmetic model filter.
`outputs/experiments/ai_profile_real/ai_profile_suite_report.json`	Auxiliary AI-profile result	Deterministic, real model-backed, and malformed raw-output profiles with raw, parsed, parse-failure, usage, cost, latency, provider, and model evidence.

7.4 Suite-Level Results

The default suite contains 17 runs. Twelve runs pass, five runs are expected ablation degradations, and no run fails or becomes blocked. This suite-level result matters because TokenShare is evaluated as a contribution system: the passing runs show that shared capacity can be converted into accepted work, while the inconclusive ablation runs show that the conversion depends on specific protections rather than on final-answer luck.

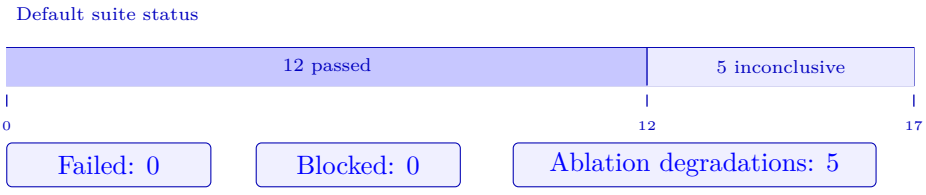

Figure 3: Suite-level status ribbon. The default suite has 17 runs: 12 passed, 5 intentional ablation degradations, 0 failed, and 0 blocked.

7.5 Experiment 1: End-to-End Factorization Lifecycle

Purpose. Experiment 1 asks whether TokenShare can turn shared bounded compute into a complete verified output. The purpose is not to benchmark factorization; factorization is a compact domain in which decomposition, execution, verification, merge, and accounting are all checkable.

Setting. The setting uses the real `factorization@0.1.0` plugin. The four cases cover direct completion, prime-range verification, semiprime decomposition, and a larger semiprime benchmark. Contributors execute bounded range-search units; the plugin owns parsing, range verification, and all-required merge.

Result and interpretation. All four cases pass with final correctness, completion rate 1.0, accounting success, and zero canonical pollution. The result shows the basic TokenShare promise: a large useful task can be split into shareable units, each unit can produce evidence, and accepted child evidence can be merged into a final result with a replayable contribution trail.

Table 4: Experiment 1: factorization lifecycle results.

Case	Instance	Output	Events	Artifacts	Interpretation
E1a	$N = 2$	[2]	22	9	Direct completion without range expansion.
E1b	$N = 97$	[97]	96	38	Prime case completed through bounded range verification.
E1c	$N = 91$	[7, 13]	96	38	Semiprime case completed through range search and all-required merge.
E1d	$N = 8051$	[83, 97]	148	62	Larger semiprime case exercised the same lifecycle at greater evidence depth.

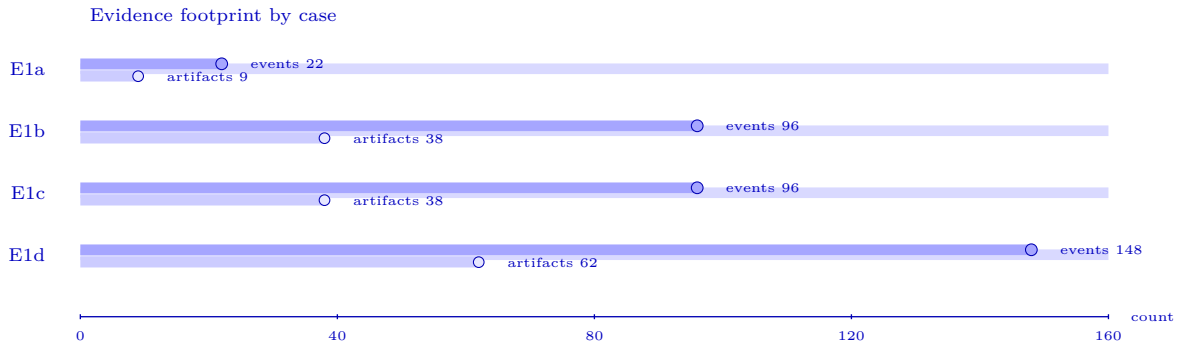

Figure 4: Lollipop-style evidence footprint for Experiment 1. E1d produces the largest event and artifact footprint, while all four cases preserve final correctness and zero canonical pollution.

The semiprime cases are the most informative because they require multiple child range results, verified canonical bindings, and a plugin-owned merge rule. They demonstrate that TokenShare is not merely recording a final answer; it is recording how distributed expenditure becomes accepted contribution.

7.6 Experiment 2: Failure Injection and Recovery

Purpose. Experiment 2 asks whether failed or malicious token expenditure can be prevented from becoming shared contribution. This is essential for TokenShare: a system that pools capacity but cannot reject bad evidence would only amplify error.

Setting. Each run injects one failure mode into the same factorization contribution lifecycle: an invalid factor, a false no-factor claim, an unparsable raw output, a worker crash or expired lease, and an over-budget no-factor recheck.

Result and interpretation. All five cases pass with final correctness, completion rate 1.0, accounting success, and zero canonical pollution. The result shows that TokenShare preserves bad expenditure as audit evidence without allowing it to become accepted contribution. Parser failures, verifier rejections, lease recovery, and budget limits are not engineering decorations; they are the safeguards that make sharing useful capacity credible.

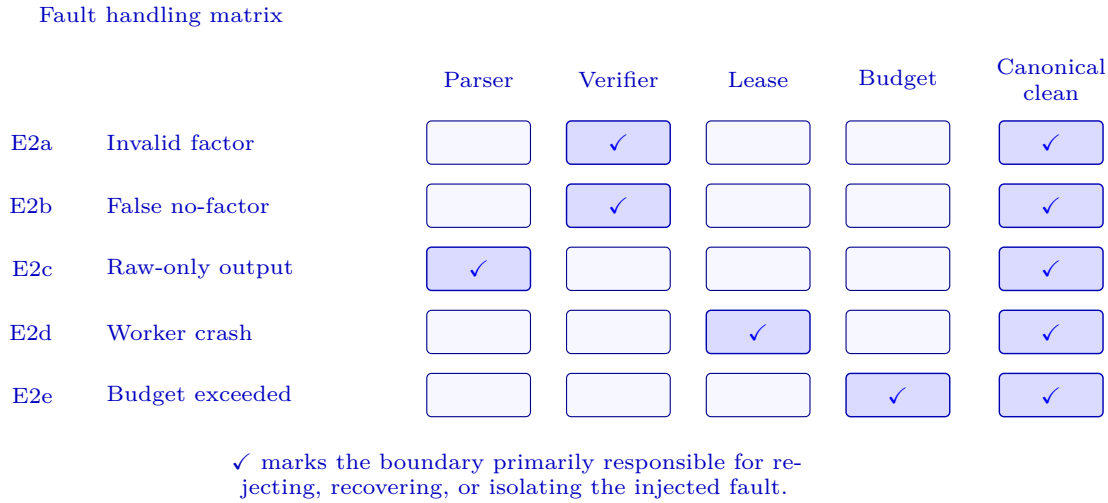

Figure 5: Failure-handling matrix for Experiment 2. Each injected failure is handled by the relevant protocol boundary, and every case preserves zero canonical pollution.

These results show that executor submission is not equivalent to contribution. A candidate output must pass the parser, verifier, and canonical-selection boundary before it can affect accepted state. Invalid evidence may remain auditable, but it does not become canonical evidence.

7.7 Experiment 3: Mechanism Ablation

Purpose. Experiment 3 asks which mechanisms are necessary for TokenShare to work as a sharing concept. If a mechanism can be removed without consequence, it is not central to the claim; if removal causes predictable degradation, it marks a real boundary in the conversion from resource expenditure to accepted contribution.

Setting. Each run disables one protection: verification, parser policy, requeue, all-required merge gate, or slot-integrity checking. The expected outcome is controlled degradation, not normal completion.

Result and interpretation. All five ablations are reported as inconclusive by design, with completion rate 0.0 and no accounting success. The verifier ablation additionally produces one canonical pollution event, showing the direct risk of bypassing plugin verification. This establishes that TokenShare depends on a bundle of boundaries: parser ownership, verifier authority, lease recovery, merge readiness, and slot integrity.

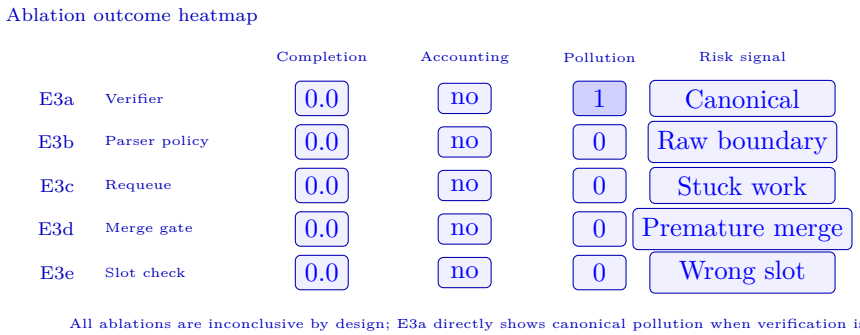

Figure 6: Mechanism-ablation heatmap. Removing core protections prevents normal completion; removing verification additionally creates canonical pollution.

The ablation suite supports a mechanism-level claim: useful sharing is not a property of the task solver alone, but of the solver together with parser ownership, verification, lease recovery, merge gating, and slot integrity.

7.8 Experiment 4: Cross-Plugin Generality

Purpose. Experiment 4 asks whether TokenShare is a general concept or merely a factorization workflow. A credible sharing mechanism should let different domains define their own valid contributions while using the same coordination and evidence lifecycle.

Setting. The experiment runs three real-plugin cases: a factorization semiprime lifecycle, a Lean direct proof, and a Lean decomposition/merge proof. The Lean cases use real checker evidence from a fixed local Lean/lake environment, including proof artifacts, logs, and environment references.

Result and interpretation. The factorization case and both Lean proof cases pass. This result supports the cross-domain claim: TokenShare can coordinate different forms of tokenized capacity, from bounded arithmetic compute to formal proof attempts, while keeping domain semantics inside plugins and keeping contribution evidence in a shared protocol lifecycle.

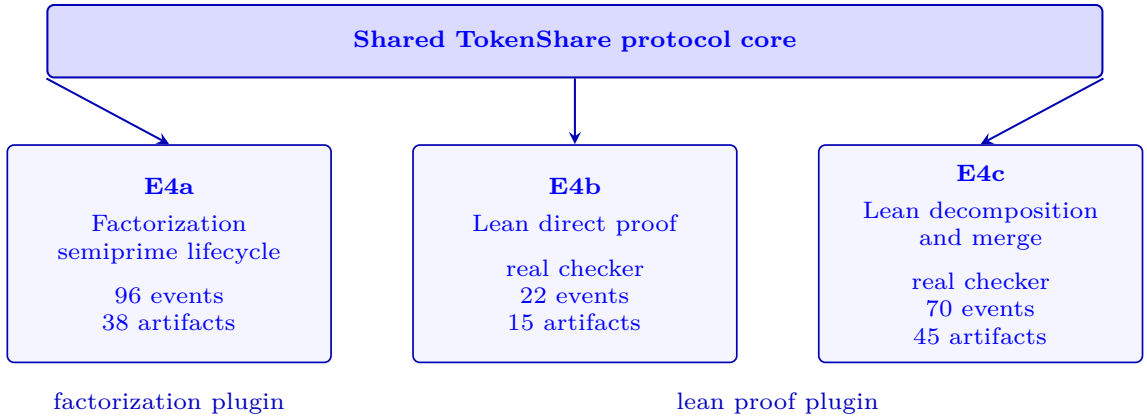

Figure 7: Cross-plugin generality in Experiment 4. The same TokenShare contribution lifecycle supports both factorization and Lean proof plugins while domain semantics remain plugin-owned.

Table 5: Experiment 4: cross-plugin generality results.

Case	Workload and plugin	Evidence footprint	Interpretation
E4a	**Semiprime lifecycle** Plugin: factorization	Checker evidence: no Events: 96 Artifacts: 38	The arithmetic-search plugin completes the shared TokenShare lifecycle through plugin-owned factorization semantics.
E4b	**Lean direct proof** Plugin: lean proof	Checker evidence: yes Events: 22 Artifacts: 15	The direct proof path is accepted by a real Lean checker, showing that protocol acceptance can rely on external formal verification evidence.
E4c	**Lean decomposition and merge** Plugin: lean proof	Checker evidence: yes Events: 70 Artifacts: 45	The split child proofs and the merged root proof are accepted by the checker, demonstrating that the protocol can coordinate decomposition and merge beyond direct proof submission.

The result supports the TokenShare generality claim. The common system records lifecycle state, artifacts, verification reports, canonical bindings, and accounting records, while factorization arithmetic and Lean proof checking remain outside the protocol core. TokenShare shares capacity and evidence; each domain still defines what valid work means.

7.9 Auxiliary Simulation: Heterogeneous Contributors

The auxiliary simulation connects the experiments back to the motivating idea of sharing tokens. It is not a fifth experiment; it approximates a multi-user setting by running the same semiprime task under three contributor profiles: a deterministic local contributor, a real model-backed contributor, and a malformed raw-output contributor. The workload is $N = 91$, with expected factors $[7, 13]$.

The simulation tests whether different resource media can enter TokenShare through the same contribution boundary. The model-backed contributor spends real API/model capacity, but the protocol does not directly trust raw responses. The malformed profile models a contributor whose expenditure produces evidence but no accepted contribution. Thus the simulation distinguishes resource spending from useful sharing: only parsed and verified evidence can count.

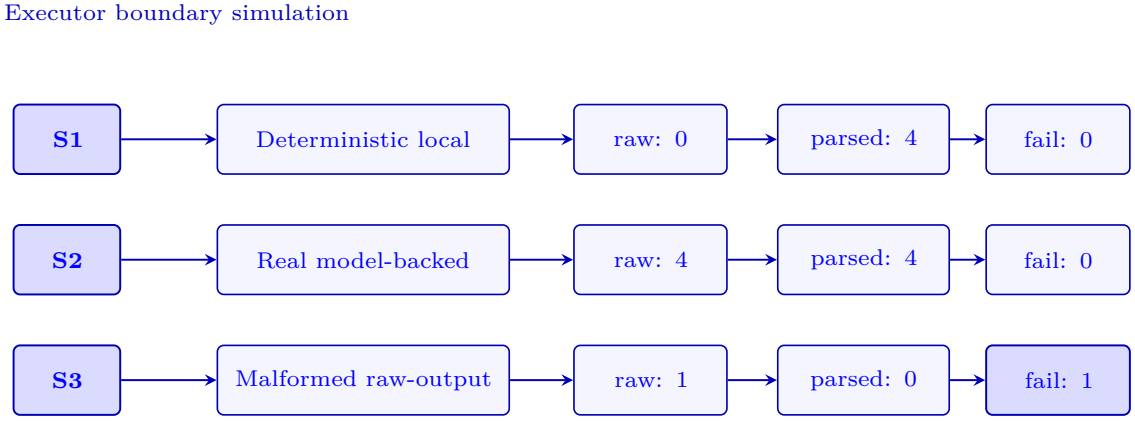

Figure 8: Executor-boundary outcomes in the contributor-heterogeneity simulation. The malformed contributor is recorded but not accepted.

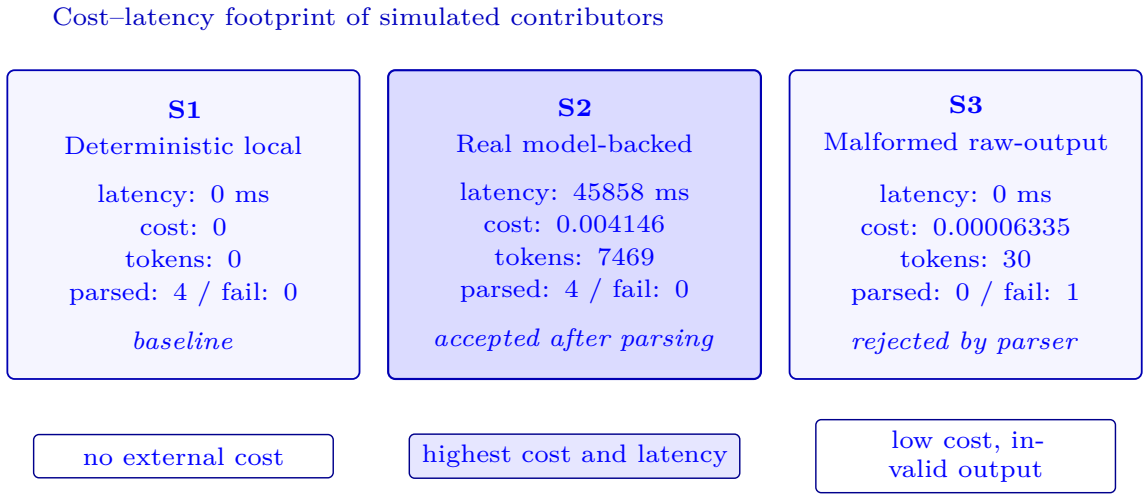

Figure 9: Cost–latency footprint of simulated contributor profiles. The model-backed contributor increases cost and latency, while the malformed contributor remains cheap but produces no accepted parsed output.

Relative to the deterministic local profile, the real model-backed profile shows no loss of task correctness, parser reliability, or retry behavior. Both profiles complete the task correctly, both produce four parsed outputs, both record

zero parse failures, and the model-backed run requires no extra retry:

$$\begin{aligned} \text{correct}_{\text{local}} &= \text{correct}_{\text{model}} = 1, \\ \text{parsed}_{\text{local}} &= \text{parsed}_{\text{model}} = 4, \\ \text{failures}_{\text{local}} &= \text{failures}_{\text{model}} = 0, \\ \Delta_{\text{retry}} &= 0. \end{aligned}$$

The real model-backed path introduces external-resource overhead in the explicit `-real-transport` AI profile run reported in `outputs/experiments/ai_profile_real`:

$$\Delta_{\text{cost}} = 0.004146, \qquad \Delta_{\text{latency}} = 45858 \text{ ms}.$$

This simulation supports the token-sharing interpretation of the protocol: contributors may differ in execution medium, reliability, and cost, but their outputs are mediated by a common parser, verifier, artifact, and accounting interface. Real model tokens can be shared into the task, yet they become useful only after the same evidence boundary as local compute.

7.10 Audit Footprint

The default suite records 1215 events and 510 artifacts. This footprint is not only execution telemetry; it is the evidence trail that lets TokenShare distinguish spent capacity from accepted contribution. A contribution economy without provenance collapses into trust in final answers. TokenShare instead keeps the intermediate requests, submissions, parser outcomes, verification reports, canonical bindings, merge records, and accounting facts inspectable.

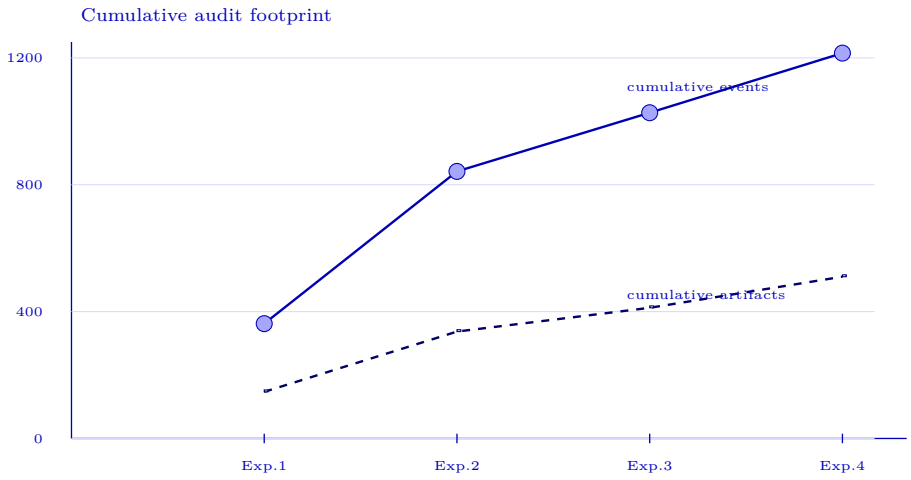

Figure 10: Cumulative audit footprint across Experiments 1–4. The monotone increase reflects accumulated replay and artifact evidence, not merely final-answer records.

Table 6: Event and artifact footprint by experiment group.

Group	Runs	Passed	Events	Artifacts
Experiment 1	4	4	362	147
Experiment 2	5	5	480	190
Experiment 3	5	0	185	75
Experiment 4	3	3	188	98
Total	17	12	1215	510

7.11 Limitations

The results provide prototype-scale evidence for the TokenShare concept rather than a large-scale performance benchmark. The factorization tasks are fixed small integers, the default deterministic suite uses one seed, and the contributor-heterogeneity simulation uses a small number of executor profiles rather than a large population of real users. Accordingly, the evidence supports the conceptual claims that tokenized capacity can be decomposed into useful units, failed contributions can be isolated, key sharing mechanisms are necessary, and the lifecycle can cross task domains. Broader throughput scaling, stronger adversarial workloads, richer replay tooling, and multi-provider cost optimization remain separate evaluation targets.

7.12 Summary

The experiments support four concept-level conclusions. First, TokenShare can convert shared bounded compute into a complete verified result. Second, invalid or malformed expenditure can remain auditable without becoming contribution. Third, parser ownership, verification, requeue, merge gating, and slot integrity are necessary for useful sharing. Fourth, the same contribution lifecycle supports factorization and real Lean proof production. The auxiliary contributor simulation further shows how local compute, real model tokens, and malformed output can enter one system while only verified evidence becomes accepted contribution.

8 Related Work

Distributed task allocation and market coordination. TokenShare builds on a long line of work on allocating work among distributed agents. The Contract Net Protocol introduced a negotiation pattern in which tasks are announced, potential contractors bid, and managers award work to selected agents [25]. Multi-robot task allocation later formalized the relation among agents, tasks, utilities, costs, and assignment structure [7], while market-based coordination studied how decentralized bidding mechanisms can trade off communication cost, robustness, and solution quality [5]. These works justify treating executor selection, leases, and retry as protocol-visible allocation decisions. TokenShare differs in that the allocated object is not a single atomic job: it is a recursively expandable task tree or DAG whose leaves may require model calls, local tools, external checkers, or bounded compute, and whose contribution records become eligible only after parser, verifier, canonical-selection, and merge boundaries succeed.

Complex crowdsourcing and decentralized crowdsourcing. Human-computation systems show that complex work can be decomposed, distributed, reviewed, and recomposed. CrowdForge proposes partition, map, and reduce primitives for crowdsourcing complex work [12], and Turkomatic explores recursive task and workflow design for Mechanical Turk-style settings [13]. Blockchain-based crowdsourcing systems such as CrowdBC and decentralized HIT protocols study payment, dispute handling, and reduced reliance on centralized platforms [15, 16]. For TokenShare, the relevant lesson is that decomposition and review need durable protocol evidence. The prototype generalizes this lesson to heterogeneous intelligent work by making plugin schemas, artifact references, verification reports, canonical choices, and merge policies part of a reusable local protocol substrate rather than a fixed human-task workflow.

Volunteer and distributed execution systems. Volunteer and cluster-computing systems provide practical lessons for unreliable and heterogeneous execution. BOINC supports volunteer scientific computation over devices that differ in capability, availability, and reliability [1]. MapReduce popularized partitioned data processing with automatic scheduling, failure handling, and re-execution on commodity clusters [4]. Ray supports dynamic task graphs for AI applications [17], while Sparrow studies decentralized low-latency scheduling for short parallel tasks [21]. TokenShare adopts the engineering vocabulary of leases, heartbeats, re-execution, and reassignment, but applies it to semantic and verifiable intelligent production: a work unit must include an input schema, output schema, verifier, provenance record, and merge rule.

Blockchains, verifiable computation, and useful work. Bitcoin demonstrates that mutually untrusted participants can be coordinated through a public ledger and proof-of-work incentives [18]. Ethereum generalizes the ledger into programmable smart contracts for decentralized applications [2]. TrueBit studies

Table 7: Positioning of TokenShare relative to adjacent work.

Line of work	Main contribution	Gap addressed by TokenShare
Task allocation	Negotiated or market-based assignment among agents	Recursive task DAGs with verification-aware routing and contribution eligibility
Complex crowdsourcing	Human workflow decomposition and recomposition	Heterogeneous LLM, tool, compute, and human contributors with typed verifiers
Distributed execution	Scheduling and recovery for unreliable machines	Semantic intelligent work units with provenance, audit, and merge rules
Ledgered verification	Public commitments, verifiable evidence, and replayable state transitions	Event-sourced evidence and replayable contribution accounting for useful-work lifecycles
LLM-agent systems	Planning, tool use, role specialization, and agent communication	Fault tolerance, parser and verifier boundaries, contribution attribution, and protocol-level evidence

interactive verification for expensive off-chain computation [26]. Proof-of-useful-work systems such as Ofelimos attempt to replace artificial hash puzzles with useful optimization work [6]. This literature informs TokenShare's use of public commitments, verifiable evidence, and replayable state transitions. The prototype brings that evidence discipline into a local useful-work setting: append-only events, content-addressed artifacts, verification reports, and deterministic replay are used to study task lifecycles across plugins and executors.

LLM decomposition, tool use, and multi-agent workflows. Recent LLM research shows that complex reasoning benefits from explicit intermediate structure. Decomposed Prompting, Least-to-Most Prompting, Tree of Thoughts, and ReAct all externalize decomposition, search, tool use, or intermediate reasoning states [11, 31, 32, 34]. HuggingGPT and related tool-use systems use an LLM to plan subtasks and route them to external models or tools [24]. Multi-agent frameworks such as CAMEL, AutoGen, MetaGPT, and ChatDev further introduce role specialization, agent communication, and structured software-production workflows [8, 14, 22, 29]. These works motivate TokenShare's use of decomposition, roles, and tool-facing agents. TokenShare complements them by making allocation, parser boundaries, verification reports, canonical selection, merge, and contribution provenance explicit protocol objects rather than hidden orchestration state.

Verification-heavy intelligent tasks. Formal proof assistants provide a useful testbed because accepted outputs can be checked deterministically. The Lean mathematical library shows how a large community can maintain formalized mathematics in Lean [27]. miniF2F provides a cross-system benchmark for formal olympiad-level mathematics [33], and LeanDojo exposes Lean proof states, premises, interaction data, and benchmarks for retrieval-augmented theorem proving [30]. These works support our Lean proof-production application: TokenShare can treat a theorem, subgoal, tactic patch, error log, or auxiliary lemma as a task unit, while Lean supplies a replayable verifier.

Annotation quality and low-resource public-good NLP. Crowdsourced annotation research provides mature models for noisy contributors and uncertain labels. Dawid–Skene style aggregation estimates latent labels and worker error rates [3]; GLAD models both annotator ability and item difficulty [28]; MACE estimates annotator competence under noisy or adversarial conditions [9]; and Snorkel shows how weak supervision sources can be combined into probabilistic labels [23]. Low-resource language work also warns that public-good NLP is not merely a data-scarcity problem: language technologies are unevenly distributed across the world's languages [10], participatory projects such as Masakhane emphasize community involvement [19], and NLLB combines large-scale data construction with human-centered evaluation for many languages [20]. These lines motivate TokenShare's emphasis on provenance, uncertainty, expert escalation, and community review rather than treating generated artifacts as automatically authoritative.

9 Conclusion

This paper presented TokenShare as a way to think about shared intelligent capacity. The central claim is that model tokens, tool calls, local compute, checker time, and human attention become useful at scale only when they can be decomposed into task units, bounded by requests, preserved as evidence, verified, merged, and counted as contributions.

The prototype uses distributed-systems mechanisms to make this concept concrete. Task DAGs create shareable units; leases and attempts bind resource expenditure to task identity; artifacts preserve evidence; plugins decide what counts as valid work; canonical selection and merge convert accepted evidence into parent progress; accounting records explain why a contribution counted. These mechanisms are not the end goal of TokenShare, but they are what make the concept credible.

The experiments provide initial evidence for the concept. Factorization shows that shared bounded compute can produce a verified result; failure injection shows that bad expenditure need not pollute accepted state; ablation shows which boundaries are necessary; Lean proof production shows that the lifecycle is not tied to one domain; and the AI contributor simulation shows how real model tokens can enter the system without bypassing parser and verifier authority. The scope of the evidence is deliberately prototype-scale: it establishes the contribution lifecycle and its necessary boundaries, while leaving benchmark breadth, contributor diversity, and resource-aware policy optimization outside the claims of this paper.

Ethical considerations

The current experiments use arithmetic and formal-proof fixtures, but broader motivating domains such as annotation, knowledge organization, and low-resource language work can involve sensitive data, community knowledge, cultural context, and identity. Extending TokenShare to those settings requires licensing review, community consent, careful data governance, expert validation, minimal disclosure, and source-specific access control. Replayable provenance should support accountability while preserving uncertainty flags and avoiding overclaiming model-generated artifacts as community-authoritative without appropriate review.

References

[1] David P. Anderson. BOINC: A platform for volunteer computing. *Journal of Grid Computing*, 18:99–122, 2020. doi: 10.1007/s10723-019-09497-9. URL https://doi.org/10.1007/s10723-019-09497-9.

[2] Vitalik Buterin. Ethereum: A next-generation smart contract and decentralized application platform, 2014. URL https://ethereum.org/en/whitepaper/.

[3] A. P. Dawid and A. M. Skene. Maximum likelihood estimation of observer error-rates using the em algorithm. *Applied Statistics*, 28(1):20–28, 1979. doi: 10.2307/2346806. URL https://doi.org/10.2307/2346806.

[4] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the Sixth Symposium on Operating System Design and Implementation*, pages 137–150, 2004. URL https://www.usenix.org/conference/osdi-04/mapreduce-simplified-data-processing-large-clusters.

[5] M. Bernardine Dias, Robert Zlot, Nidhi Kalra, and Anthony Stentz. Market-based multirobot coordination: A survey and analysis. *Proceedings of the IEEE*, 94(7):1257–1270, 2006. doi: 10.1109/JPROC.2006.876939. URL https://doi.org/10.1109/JPROC.2006.876939.

[6] Matthias Fitzi, Aggelos Kiayias, Giorgos Panagiotakos, and Alexander Russell. Ofelimos: Combinatorial optimization via proof-of-useful-work: A provably secure blockchain protocol. In *Advances in Cryptology - CRYPTO 2022*, volume 13508 of *Lecture Notes in Computer Science*, pages 339–369. Springer, 2022. doi: 10.1007/978-3-031-15979-4_12. URL https://doi.org/10.1007/978-3-031-15979-4_12.

[7] Brian P. Gerkey and Maja J. Mataric. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004. doi: 10.1177/0278364904045564. URL https://doi.org/10.1177/0278364904045564.

[8] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VtmBAGCN7o.

[9] Dirk Hovy, Taylor Berg-Kirkpatrick, Ashish Vaswani, and Eduard Hovy. Learning whom to trust with mace. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1120–1130, 2013. URL https://aclanthology.org/N13-1132/.

[10] Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6282–6293, 2020. doi: 10.18653/v1/2020.acl-main.560. URL https://aclanthology.org/2020.acl-main.560/.

[11] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=_nGgzQjzaRy.

[12] Aniket Kittur, Boris Smus, Susheel Khamkar, and Robert E. Kraut. Crowdforge: Crowdsourcing complex work. In *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology*, pages 43–52, 2011. doi: 10.1145/2047196.2047202. URL https://doi.org/10.1145/2047196.2047202.

[13] Anand P. Kulkarni, Matthew Can, and Bjoern Hartmann. Turkomatic: Automatic recursive task and workflow design for mechanical turk. In *CHI Extended Abstracts on Human Factors in Computing Systems*, pages 2053–2058, 2011. doi: 10.1145/1979742.1979827. URL https://doi.org/10.1145/1979742.1979827.

[14] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for mind exploration of large language model society, 2023. URL https://arxiv.org/abs/2303.17760.

[15] Ming Li, Jian Weng, Anjia Yang, Wei Lu, Yue Zhang, Lin Hou, Jia-Nan Liu, Yang Xiang, and Robert H. Deng. CrowdBC: A blockchain-based decentralized framework for crowdsourcing. *IEEE Transactions on Parallel and Distributed Systems*, 30(6):1251–1266, 2019. doi: 10.1109/TPDS.2018.2881735. URL https://doi.org/10.1109/TPDS.2018.2881735.

[16] Yihuai Liang, Yan Li, and Byeong-Seok Shin. Decentralized crowdsourcing for human intelligence tasks with efficient on-chain cost. *Proceedings of the VLDB Endowment*, 15(9):1875–1888, 2022. doi: 10.14778/3538598.3538609. URL https://www.vldb.org/pvldb/vol15/p1875-shin.pdf.

[17] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, pages 561–577, 2018. URL https://www.usenix.org/conference/osdi18/presentation/moritz.

[18] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008. URL https://bitcoin.org/bitcoin.pdf.

[19] Wilhelmina Nekoto, Vukosi Marivate, Tshinondiwa Matsila, Timi Fasubaa, Tajudeen Kolawole, Taiwo Fagbohungbe, Solomon Oluwole Akinola, Shamsuddee Hassan Muhammad, Salomon Kabongo, Salomey Osei, et al. Participatory research for low-resourced machine translation: A case study in

african languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2144–2160, 2020. doi: 10.18653/v1/2020.findings-emnlp.195. URL https://aclanthology.org/2020.findings-emnlp.195/.

[20] NLLB Team, Marta R. Costa-jussa, James Cross, Onur Celebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, et al. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*, 2022. doi: 10.48550/arXiv.2207.04672. URL https://arxiv.org/abs/2207.04672.

[21] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. Sparrow: Distributed, low latency scheduling. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 69–84, 2013. doi: 10.1145/2517349.2522716. URL https://doi.org/10.1145/2517349.2522716.

[22] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development, 2023. URL https://arxiv.org/abs/2307.07924.

[23] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Re. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017. doi: 10.14778/3157794.3157797. URL https://doi.org/10.14778/3157794.3157797.

[24] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face, 2023. URL https://arxiv.org/abs/2303.17580.

[25] Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516. URL https://doi.org/10.1109/TC.1980.1675516.

[26] Jason Teutsch and Christian Reitwiessner. A scalable verification solution for blockchains, 2019. URL https://arxiv.org/abs/1908.04756.

[27] The mathlib Community. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020. doi: 10.1145/3372885.3373824. URL https://doi.org/10.1145/3372885.3373824.

[28] Jacob Whitehill, Ting-fan Wu, Jacob Bergsma, Javier R. Movellan, and Paul L. Ruvolo. Whose vote should count more: Optimal integration of labels from labelers of unknown expertise. In *Advances in Neural Information Processing Systems*, volume 22, 2009. URL https://papers.nips.cc/paper/2009/hash/f899139df5e1059396431415e770c6dd-Abstract.html.

[29] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

[30] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2306.15626.

[31] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2305.10601.

[32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[33] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal olympiad-level mathematics, 2021. URL https://arxiv.org/abs/2109.00110.

[34] Denny Zhou, Nathanael Scharli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WZH7099tgfM.